

OOT_重构

答题模板

1. 指出 Bad Smell

这段代码存在 `XXX` 的坏味道，表现为.....

2. 说明问题

这种设计会导致职责不清、重复修改、难以扩展、违反 SRP/OCP/封装等问题。

3. 给出重构方案

可以使用 `Extract Method`、`Extract Class`、`Move Method`、`Replace Conditional with Polymorphism`、`Introduce Parameter Object` 等方式重构。

4. 必要时画 UML

如果能重构到某个设计模式，就画对应的设计模式 UML，例如 Strategy、Observer、Adapter、Composite、Decorator、Command、DI 等。

1. Duplicated Code 重复代码

小题背景

订单系统中，`EmailReceiptPrinter` 和 `SmsReceiptPrinter` 都需要计算订单总价、格式化订单编号和日期。两处代码几乎一样，只是最后发送方式不同。

Bad Java Code

```
class EmailReceiptPrinter {
    public void print(Order order) {
        double total = 0;
        for (OrderItem item : order.getItems()) {
            total += item.getPrice() * item.getQuantity();
        }

        String title = "Order #" + order.getId();
        String date = order.getDate().toString();

        String receipt = title + "\n" + date + "\nTotal: " + total;
        sendEmail(order.getCustomerEmail(), receipt);
    }

    private void sendEmail(String email, String text) {
        // send email
    }
}
```

```

    }
}

class SmsReceiptPrinter {
    public void print(Order order) {
        double total = 0;
        for (OrderItem item : order.getItems()) {
            total += item.getPrice() * item.getQuantity();
        }

        String title = "Order #" + order.getId();
        String date = order.getDate().toString();

        String receipt = title + "\n" + date + "\nTotal: " + total;
        sendSms(order.getCustomerPhone(), receipt);
    }

    private void sendSms(String phone, String text) {
        // send sms
    }
}

```

指出 Bad Smell

这里存在 **Duplicated Code**。

两处 `print()` 中都有相同的“计算总价”和“生成收据文本”的逻辑，只是最后发送渠道不同。

为什么不好

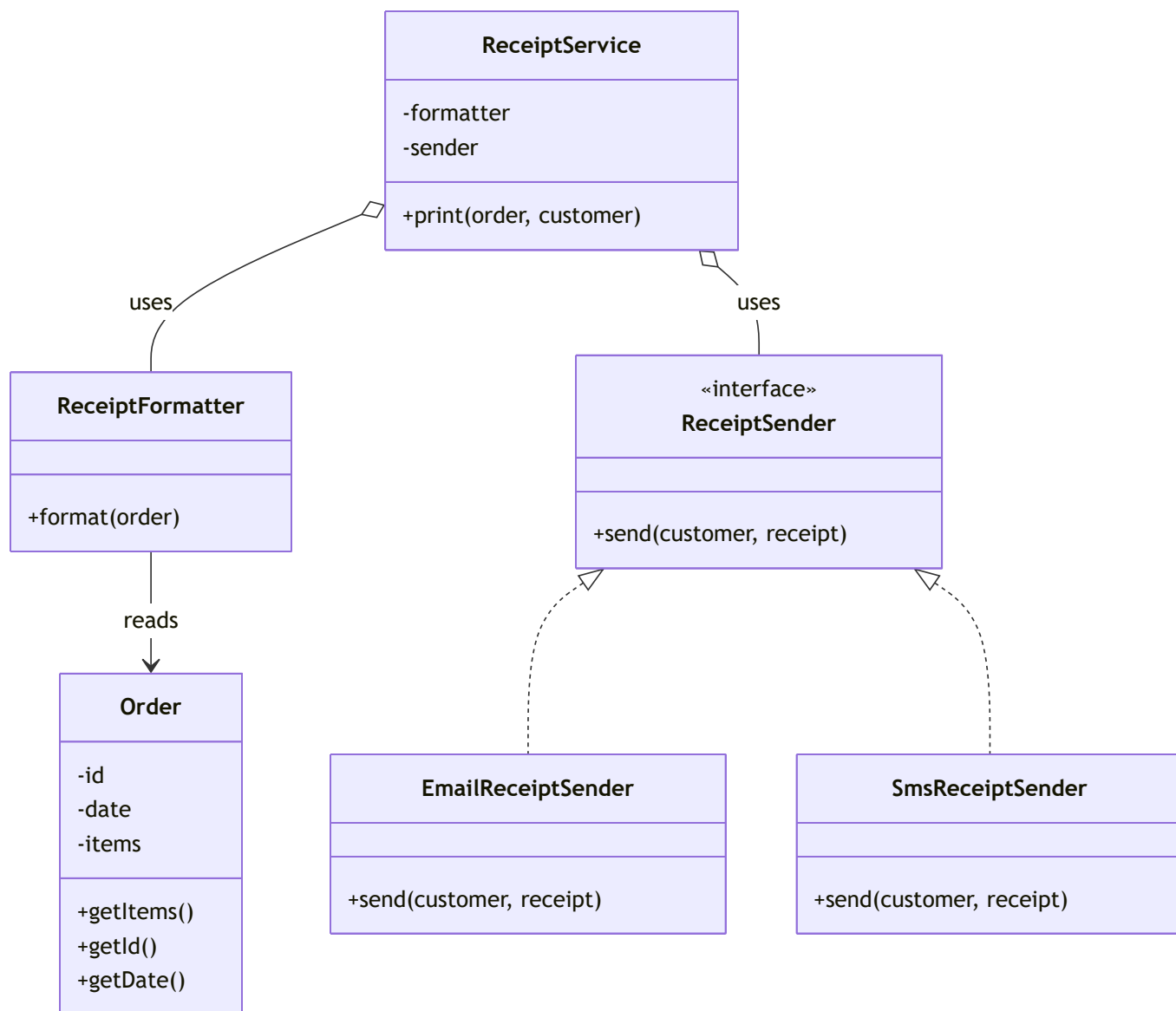
重复代码会导致平行修改：以后收据格式变化时，必须同时修改多个类。如果漏改其中一个类，就会产生不一致。课件中也强调，重复代码往往意味着设计没有被充分分解。

重构方案

使用 **Extract Class / Extract Method**。

把公共的收据生成逻辑提取到 `ReceiptFormatter` 中。发送方式由不同 Sender 负责。

重构后 Mermaid UML



答题文字模板

该代码存在 Duplicated Code。两个类中都有相同的计算总价和格式化收据逻辑，导致以后修改收据格式时需要同时修改多个地方，容易出现不一致。可以将公共逻辑提取到 `ReceiptFormatter` 中，将变化的发送方式抽象为 `ReceiptSender` 接口。这样公共逻辑只有一份，发送方式可以独立扩展。

2. Long Method / Deeply Nested Loop 长方法与深层嵌套

小题背景

学校成绩系统需要统计一个班级的及格学生数量。当前方法中包含数据过滤、异常判断、统计、打印报告等多个步骤，并且嵌套很深。

Bad Java Code

```

class GradeReportService {
    public void printPassReport(List<Student> students) {
        int passCount = 0;
    }
}
  
```

```

    for (Student s : students) {
        if (s != null) {
            if (s.isActive()) {
                if (s.getScores() != null) {
                    double sum = 0;
                    int count = 0;

                    for (Score score : s.getScores()) {
                        if (score != null) {
                            if (score.getValue() >= 0) {
                                sum += score.getValue();
                                count++;
                            }
                        }
                    }

                    if (count > 0) {
                        double avg = sum / count;
                        if (avg >= 60) {
                            passCount++;
                            System.out.println(s.getName() + " passed");
                        }
                    }
                }
            }
        }
    }

    System.out.println("Pass count: " + passCount);
}

```

指出 Bad Smell

这里存在 **Long Method** 和 **Deeply Nested Loop / Deeply Nested Conditional**。
一个方法做了太多事，而且嵌套层级过深。

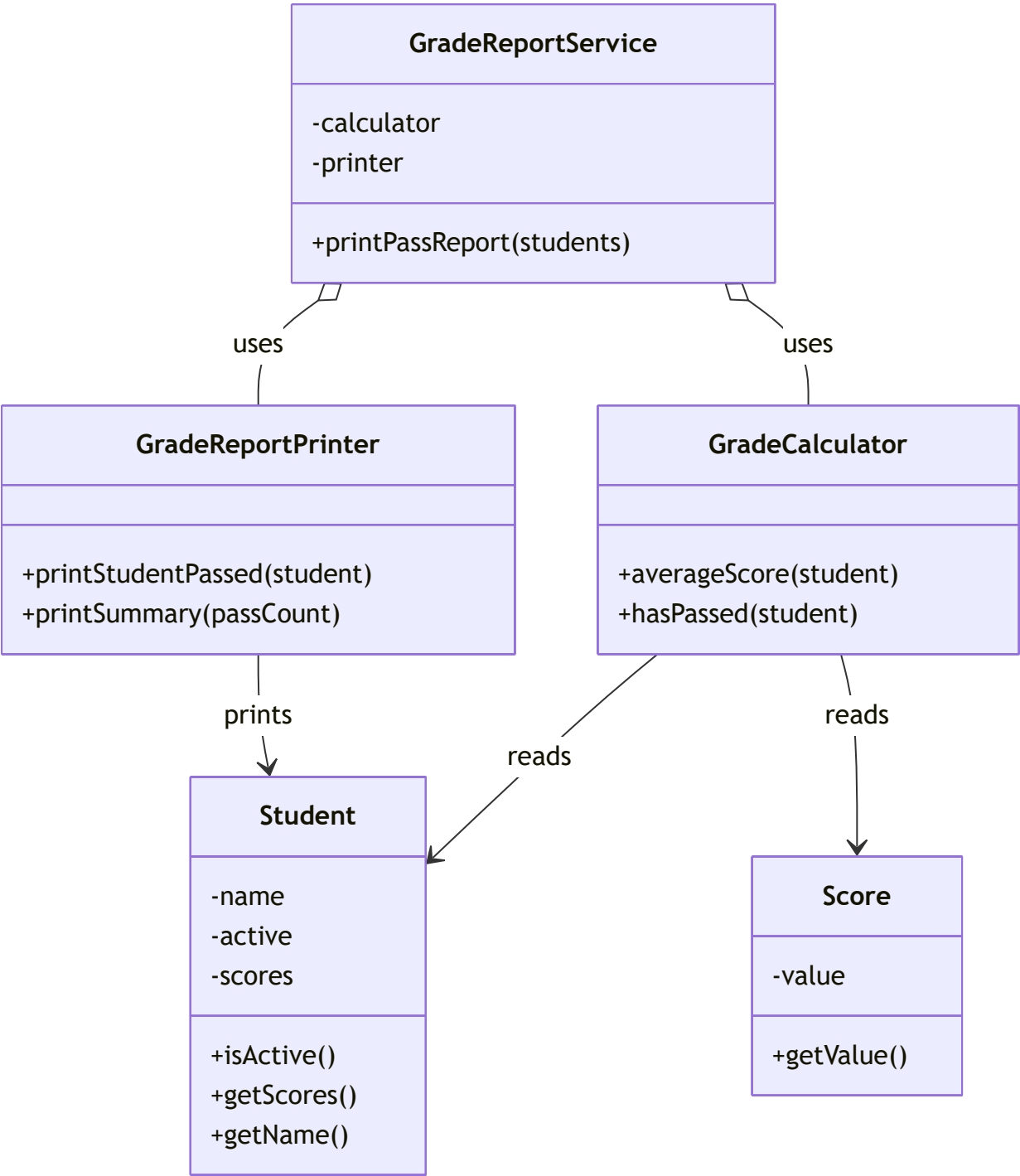
为什么不好

方法太长会降低可读性，也不利于测试。深层嵌套会让逻辑难以理解，修改时容易破坏原有行为。课件中提到，过长例程和过深循环都应当通过提取小例程降低复杂度。

重构方案

使用 **Extract Method** 和 **Guard Clause**。
把“学生是否有效”“计算平均分”“打印报告”等步骤提取为独立方法。

重构后 Mermaid UML



答题文字模板

该方法存在 Long Method 和深层嵌套的坏味道。它同时负责过滤学生、计算平均分、判断是否及格和打印报告，职责过多。可以使用 Extract Method，将平均分计算、及格判断、报告输出分别提取到独立方法或类中。重构后每个方法只做一件事，代码更容易理解和测试。

3. Public Data Members 公共数据成员

小题背景

排课系统中，`Room` 类把容量、位置、是否可用等字段全部公开。其他类可以任意修改这些字段，导致对象状态不可控。

Bad Java Code

```
class Room {
    public String roomId;
    public int capacity;
    public boolean available;
}

class BookingService {
    public void book(Room room, int studentCount) {
        if (room.available && room.capacity >= studentCount) {
            room.available = false;
        }
    }
}
```

指出 Bad Smell

这里存在 **Public Data Members**。
类的数据成员直接暴露给外部，破坏封装。

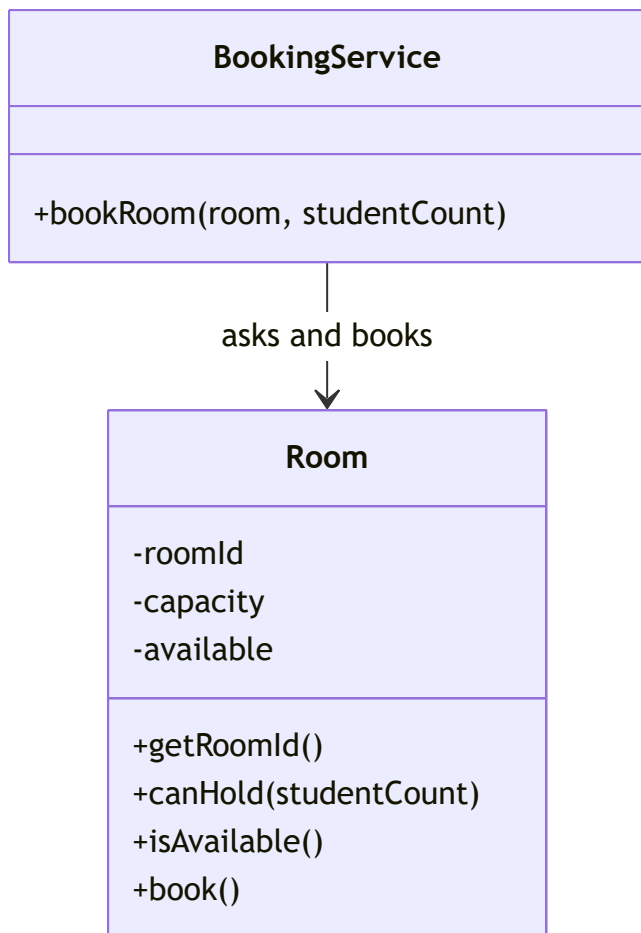
为什么不好

外部代码可以随意修改对象内部状态，例如把 `capacity` 改成负数，或者绕过预约规则直接修改 `available`。这会让类失去对自身不变量的控制。课件中明确指出，公共数据成员会模糊接口和实现的边界，违反封装。

重构方案

使用 **Encapsulate Field**。
将字段改为 `private`，通过方法控制访问和修改。

重构后 Mermaid UML



答题文字模板

该代码存在 Public Data Members 的坏味道。`Room` 的字段被外部直接访问和修改，破坏封装，使对象状态不受控制。可以使用 Encapsulate Field，将字段设为 `private`，并通过 `canHold()`、`book()` 等方法表达业务规则。这样可以保护内部状态，并使接口更加清晰。

4. Long Parameter List / Data Clumps 参数列表过长与数据泥团

小题背景

银行系统中创建转账交易时，每个方法都要传入客户编号、来源账户、目标账户、金额、币种、日期、备注等参数。这些参数经常一起出现。

Bad Java Code

```
class TransferService {
    public void transfer(
        String customerId,
        String fromAccount,
        String toAccount,
        double amount,
        String currency,
```

```
        String date,
        String note
    ) {
        // validate transfer
        // execute transfer
        // save history
    }
}
```

指出 Bad Smell

这里存在 **Long Parameter List** 和 **Data Clumps**。

多个相关参数总是一起出现，说明它们应该被组织成一个对象。

为什么不好

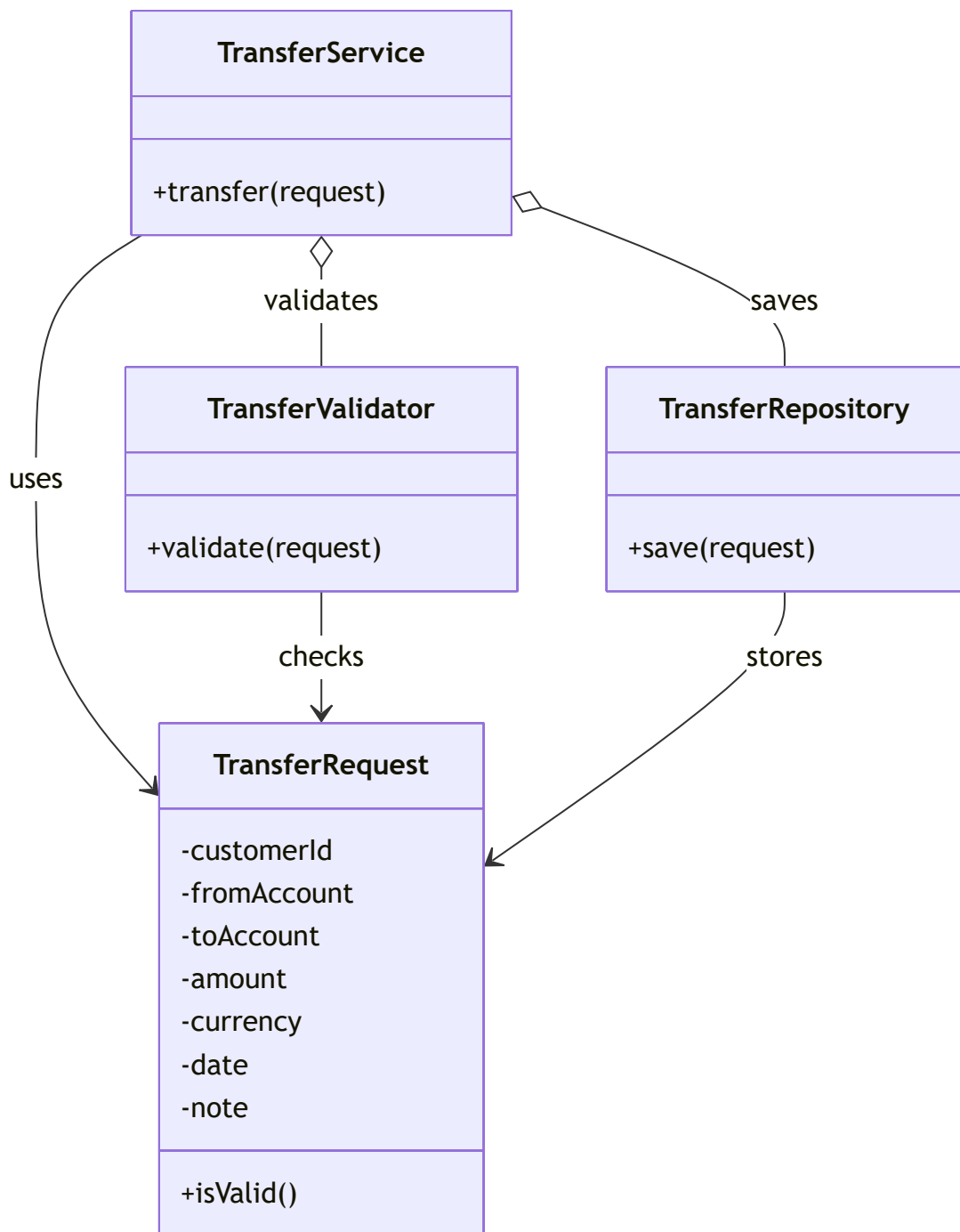
长参数列表难以阅读，调用时容易传错顺序。例如 `fromAccount` 和 `toAccount` 都是 `String`，编译器无法帮助发现传反的问题。课件中也指出，过长参数列表说明方法接口抽象不清晰。

重构方案

使用 **Introduce Parameter Object**。

把这些相关参数封装为 `TransferRequest`。

重构后 Mermaid UML



答题文字模板

该代码存在 Long Parameter List 和 Data Clumps。多个相关数据总是一起传递，说明它们构成了一个独立概念。可以使用 Introduce Parameter Object，将这些参数封装为 `TransferRequest`。这样方法签名更清晰，也可以把校验逻辑移动到请求对象或验证器中。

5. Primitive Obsession 基本类型偏执

小题背景

电商系统中所有金额都用 `double` 表示，所有币种都用 `String` 表示。不同模块直接对金额做计算，没有统一的金额校验和格式化规则。

Bad Java Code

```
class Order {
    private double totalPrice;
    private String currency;

    public Order(double totalPrice, String currency) {
        this.totalPrice = totalPrice;
        this.currency = currency;
    }

    public double getTotalPrice() {
        return totalPrice;
    }

    public String getCurrency() {
        return currency;
    }
}

class PaymentService {
    public void pay(double amount, String currency) {
        if (amount < 0) {
            throw new RuntimeException("invalid amount");
        }
        // pay money
    }
}
```

指出 Bad Smell

这里存在 **Primitive Obsession / A primitive data type is overloaded**。

用 `double` 和 `String` 表示业务概念“钱”，导致业务规则分散。

为什么不好

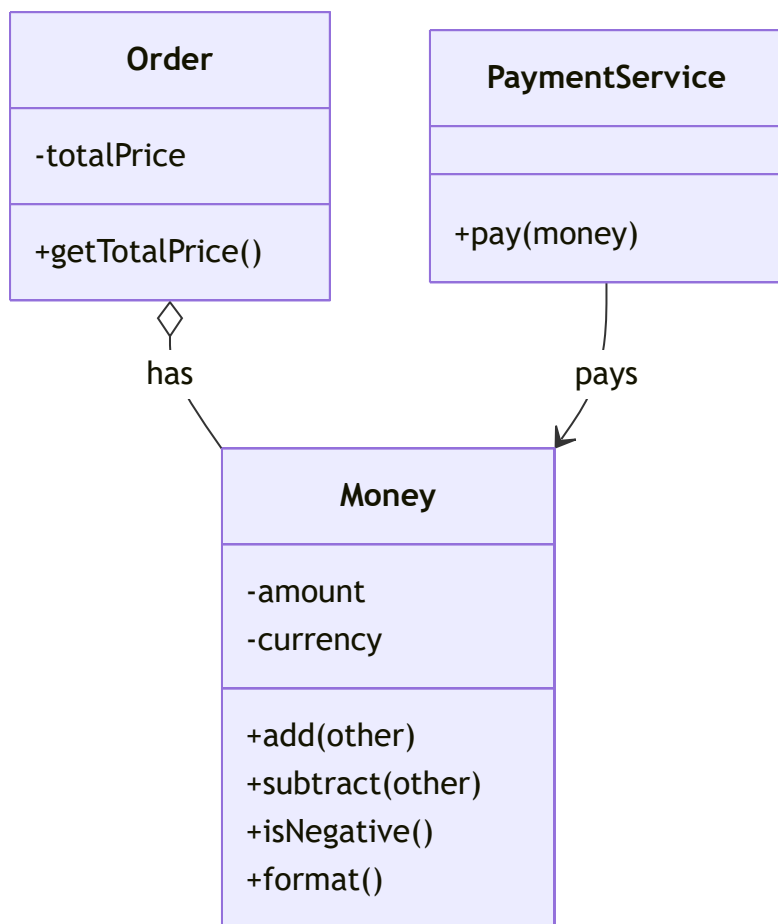
金额不只是一个数字，还包含币种、非负约束、加减规则、格式化规则等。用基本类型表示会让这些规则散落在各处，也容易把金额和其他数字混用。课件中提到，如果用整数表示 Money 和 Temperature，编译器不会阻止错误赋值。

重构方案

使用 **Replace Primitive with Object / Value Object**。

创建 `Money` 类封装金额和币种。

重构后 Mermaid UML



答题文字模板

该代码存在 Primitive Obsession。金额被简单地表示为 `double` 和 `String`，导致金额相关规则分散在多个类中。可以创建 `Money` 值对象，把金额、币种、合法性检查和格式化逻辑封装起来。这样可以增强类型安全，避免把金额和普通数字混用。

6. Feature Envy 依恋情结

小题背景

订单系统中，`OrderService` 为了计算顾客折扣，大量访问 `Customer` 的积分、等级、注册年份等数据。这个方法明显更关心 `Customer` 的内部数据，而不是 `OrderService` 自己的数据。

Bad Java Code

```
class Customer {
    private int points;
    private String level;
    private int registerYear;

    public int getPoints() {
```

```

        return points;
    }

    public String getLevel() {
        return level;
    }

    public int getRegisterYear() {
        return registerYear;
    }
}

class OrderService {
    public double calculateDiscount(Customer customer) {
        double discount = 0;

        if (customer.getLevel().equals("VIP")) {
            discount += 0.1;
        }

        if (customer.getPoints() > 1000) {
            discount += 0.05;
        }

        if (customer.getRegisterYear() < 2020) {
            discount += 0.03;
        }

        return discount;
    }
}

```

指出 Bad Smell

这里存在 **Feature Envy**。

`OrderService.calculateDiscount()` 大量使用 `Customer` 的数据，说明这个行为可能更属于 `Customer`。

为什么不好

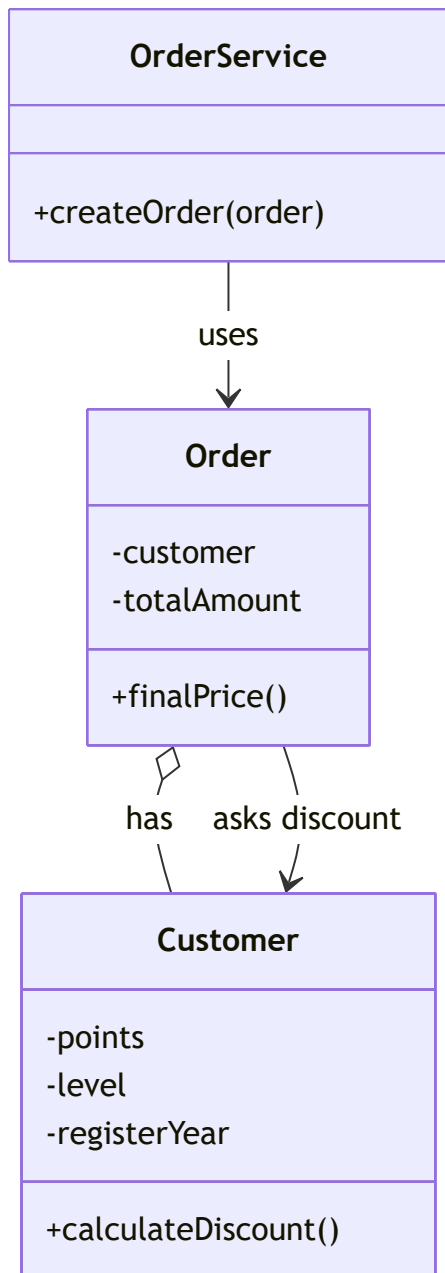
行为和数据分离会降低内聚。`Customer` 的折扣规则如果变化，需要修改 `OrderService`，而不是修改拥有这些数据的类。课件中指出，如果一个例程使用另一个类的特征比使用自己类的特征更多，通常应该把该例程移动到另一个类中。

重构方案

使用 **Move Method**。

把 `calculateDiscount()` 移动到 `Customer` 中，或提取为独立的 `DiscountPolicy`。

重构后 Mermaid UML



答题文字模板

该代码存在 Feature Envy。`calculateDiscount()` 主要使用 `Customer` 的字段和规则，因此该行为更应该属于 `Customer` 或独立的折扣策略类。可以使用 Move Method，将折扣计算移动到 `Customer` 中。这样数据和行为更集中，类的内聚性更好。

7. Large Class / Poor Cohesion 大类与低内聚

小题背景

`UserManager` 同时负责用户注册、密码加密、发送邮件、保存数据库、生成报表。每次改一个功能时都要改这个类。

Bad Java Code

```
class UserManager {  
    public void register(String email, String password) {  
        String encrypted = encryptPassword(password);  
        saveUser(email, encrypted);  
        sendWelcomeEmail(email);  
    }  
  
    public String encryptPassword(String password) {  
        return "encrypted-" + password;  
    }  
  
    public void saveUser(String email, String encryptedPassword) {  
        // save to database  
    }  
  
    public void sendWelcomeEmail(String email) {  
        // send email  
    }  
  
    public void generateUserReport() {  
        // generate report  
    }  
}
```

指出 Bad Smell

这里存在 **Large Class** 和 **Poor Cohesion**。

一个类承担了注册流程、加密、数据库、邮件、报表等多个不相关职责。

为什么不好

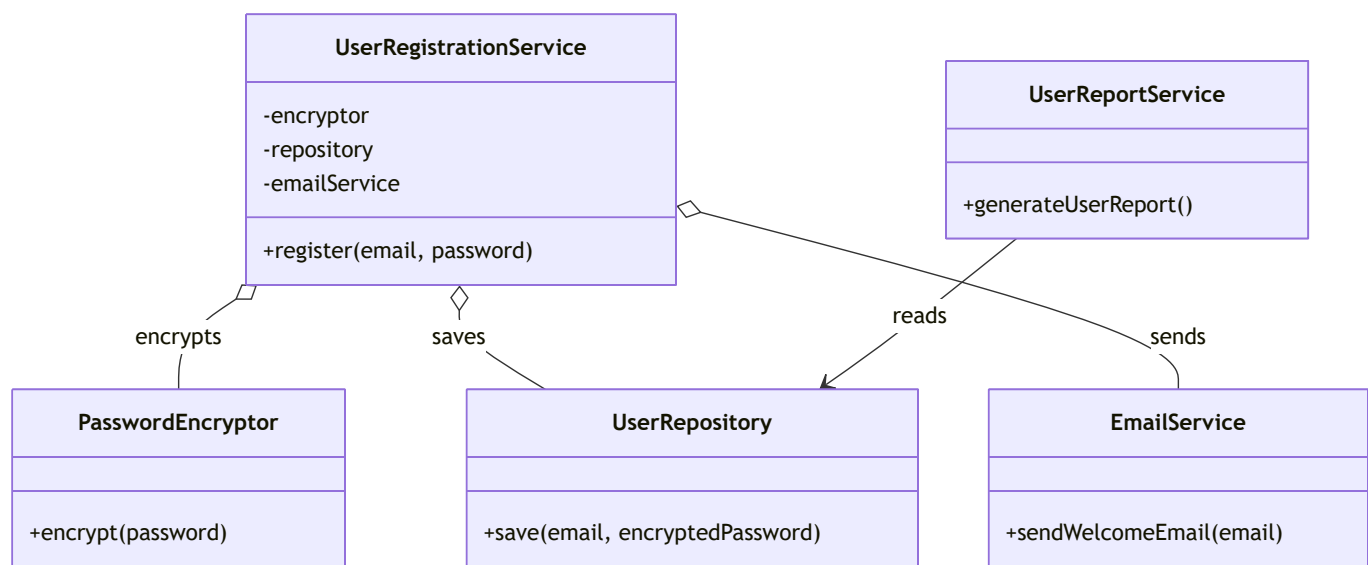
类职责太多，会导致修改原因过多，违反单一职责原则。课件中提到，如果一个类拥有很多无关责任，就应该拆分成多个具有内聚职责的类。

重构方案

使用 **Extract Class**。

拆分出 `PasswordEncryptor`、`UserRepository`、`EmailService`、`UserReportService`。

重构后 Mermaid UML



答题文字模板

该代码存在 Large Class 和 Poor Cohesion。`UserManager` 同时承担多个不相关职责，导致类过大且变化原因过多。可以使用 Extract Class，将密码加密、数据保存、邮件发送、报表生成拆分到不同类中。重构后每个类只负责一个清晰职责，符合单一职责原则。

8. Switch Statements / Parallel Case Statements 条件分支与平行修改

小题背景

物流系统根据不同配送方式计算运费。当前代码在多个地方都使用 `if / else` 判断配送类型，例如计算运费、预计送达时间、打印配送说明。每新增一种配送方式，多个方法都要修改。

Bad Java Code

```
class ShippingService {
    public double calculateFee(String type, double weight) {
        if (type.equals("AIR")) {
            return weight * 10;
        } else if (type.equals("TRUCK")) {
            return weight * 5;
        } else if (type.equals("SHIP")) {
            return weight * 2;
        }
        return 0;
    }
}
```

```
public int estimateDays(String type) {  
    if (type.equals("AIR")) {  
        return 1;  
    } else if (type.equals("TRUCK")) {  
        return 3;  
    } else if (type.equals("SHIP")) {  
        return 10;  
    }  
    return 0;  
}
```

指出 Bad Smell

这里存在 **Switch Statements / Parallel Case Statements**。

多个方法中都有类似的类型判断，每新增一个类型都要同步修改多个分支。

为什么不好

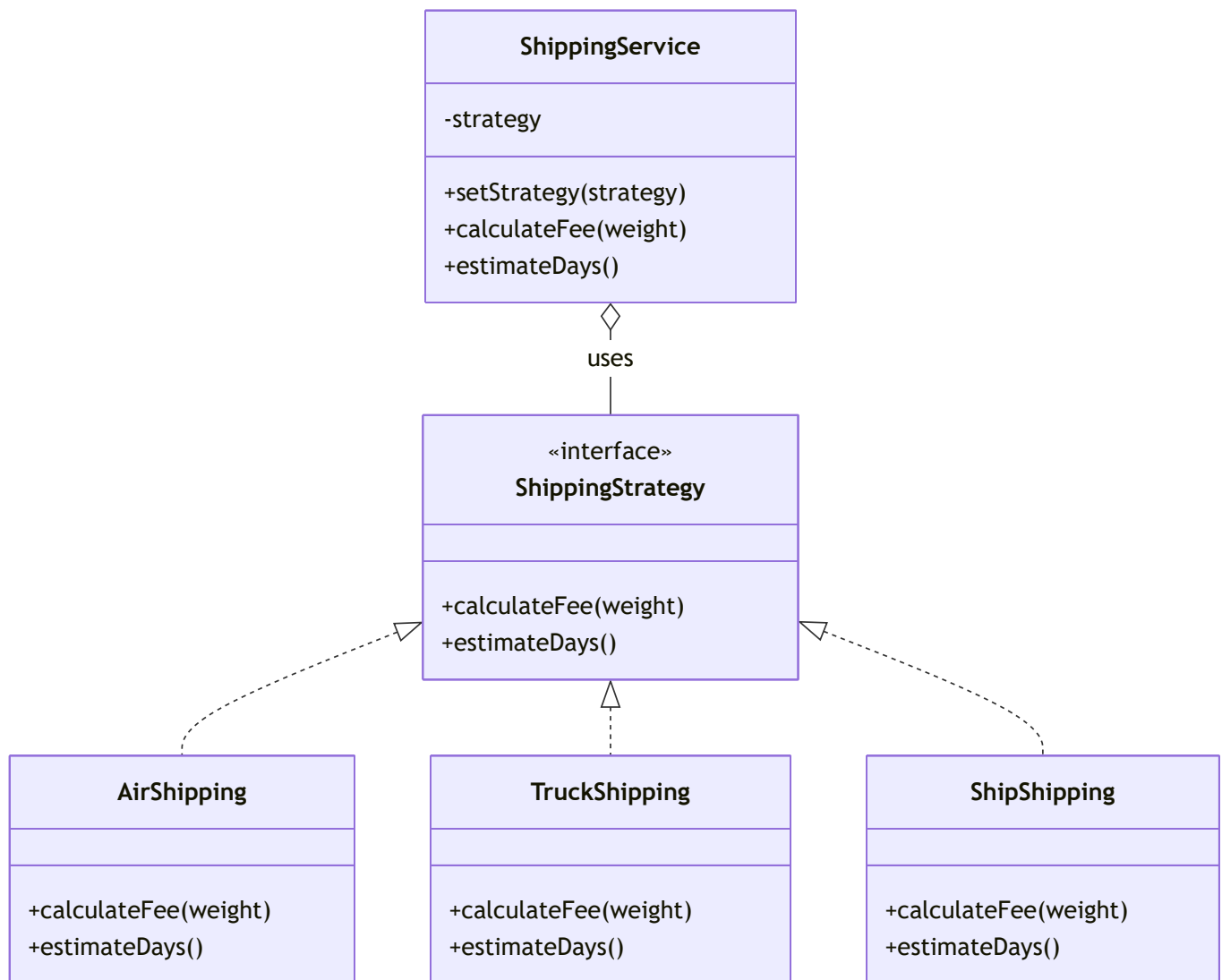
这会导致平行修改和违反开闭原则。业务变化不是通过新增类完成，而是通过修改已有条件分支完成，容易漏改。课件中提到，如果 case statements 需要被平行修改，就应该考虑用继承或多态。

重构方案

使用 **Replace Conditional with Polymorphism**，进一步可以重构为 **Strategy Pattern**。

把不同配送方式封装成不同策略类。

重构后 Mermaid UML



答题文字模板

该代码存在 Switch Statements 和 Parallel Case Statements。多个方法都根据同一个 `type` 做条件判断，每新增一种配送方式都要修改多个方法，违反开闭原则。可以使用 Replace Conditional with Polymorphism，将不同配送方式封装为实现 `ShippingStrategy` 接口的类。重构后新增配送方式只需增加新的策略类。

9. Inappropriate Intimacy 过分亲密

小题背景

`ReportPrinter` 为了打印用户地址，直接访问 `User` 的 `Profile`、`Address` 等内部对象，并拼接多个内部字段。它知道了太多 `User` 的内部结构。

Bad Java Code

```
class Address {
    String city;
    String street;
```

```

        String zipCode;
    }

    class Profile {
        Address address;
    }

    class User {
        Profile profile;
    }

    class ReportPrinter {
        public void printAddress(User user) {
            String text = user.profile.address.city + ", "
                + user.profile.address.street + ", "
                + user.profile.address.zipCode;

            System.out.println(text);
        }
    }
}

```

指出 Bad Smell

这里存在 **Inappropriate Intimacy**，同时也有 **Public Data Members**。

`ReportPrinter` 知道并依赖了 `User -> Profile -> Address` 的内部结构。

为什么不好

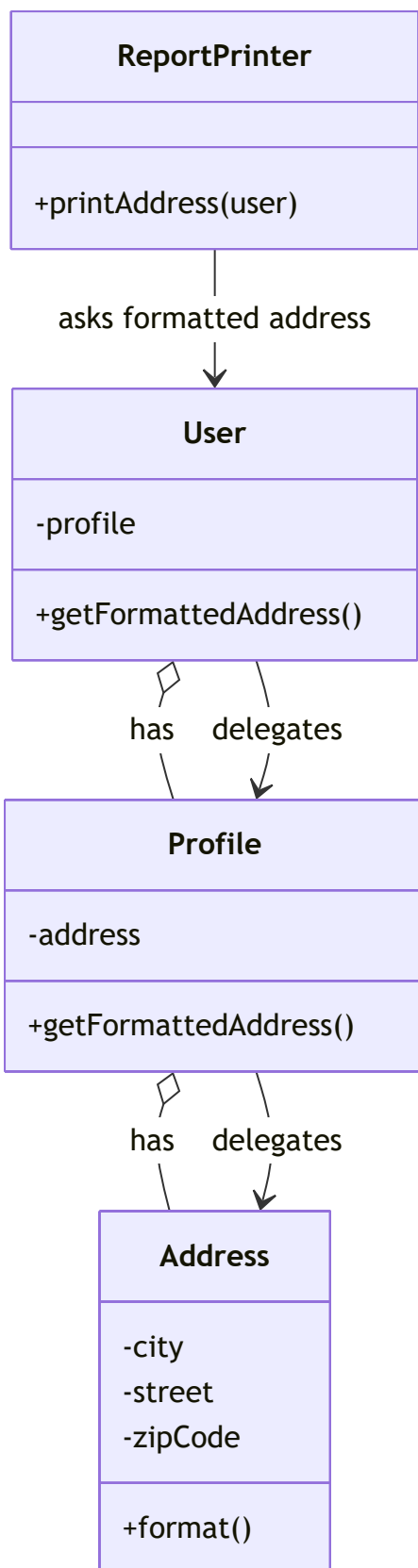
如果 `User` 的内部结构发生变化，例如 `Profile` 改名或 `Address` 拆分，`ReportPrinter` 也要修改。这破坏封装，增加了类之间的耦合。课件中强调，类之间知道彼此细节越多，越容易产生修改的涟漪效应。

重构方案

使用 **Encapsulate Field / Hide Delegate**。

让 `User` 或 `Address` 提供稳定接口，例如 `user.getFormattedAddress()`。

重构后 Mermaid UML



答题文字模板

该代码存在 Inappropriate Intimacy。`ReportPrinter` 直接访问 `User` 的多层内部结构，导致两个类耦合过强。可以使用 Encapsulate Field 和 Hide Delegate，让 `User` 提供 `getFormattedAddress()` 这样的稳定接口。这样外部类不再依赖内部对象结构，封装性更好。

10. Global Variables / Direct Concrete Dependency 全局变量与直接依赖具体类

小题背景

订单系统中，`OrderService` 直接使用全局配置对象和直接创建 `EmailSender`。这样测试时很难替换配置和发送器。

Bad Java Code

```
class GlobalConfig {
    public static String smtpServer = "smtp.example.com";
}

class EmailSender {
    public void send(String email, String message) {
        System.out.println("Send by " + GlobalConfig.smtpServer);
    }
}

class OrderService {
    public void createOrder(String email) {
        // create order
        EmailSender sender = new EmailSender();
        sender.send(email, "order created");
    }
}
```

指出 Bad Smell

这里存在 **Global Variables** 和 **Direct Concrete Dependency**。

`EmailSender` 直接依赖全局配置，`OrderService` 直接创建具体的 `EmailSender`。

为什么不好

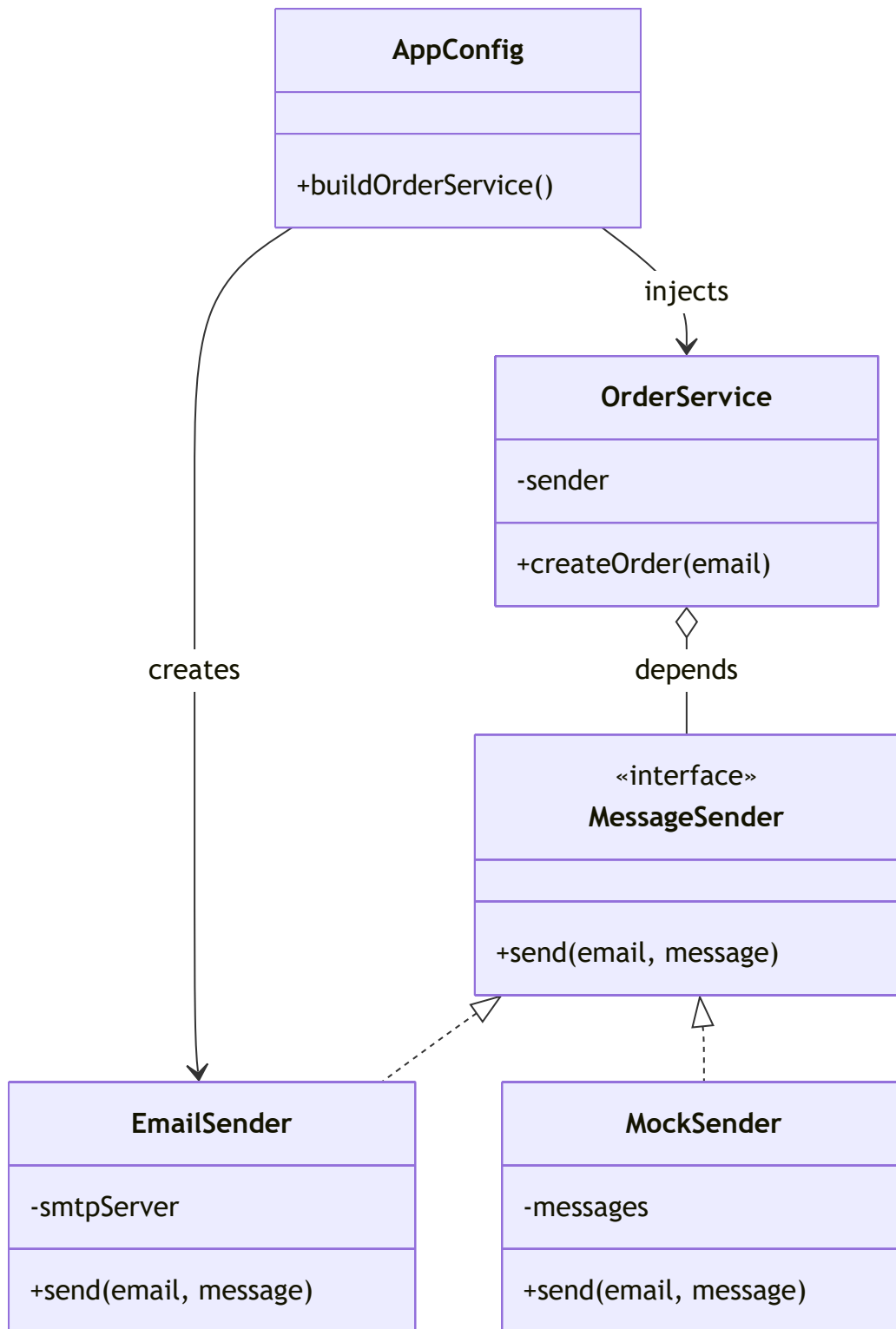
全局变量会产生隐藏依赖，任何地方都可能修改它，导致行为难以预测。直接 `new` 具体类会让业务类难以测试，因为无法替换成 Mock 对象。课件中指出使用全局变量时应重新审视设计。

重构方案

使用 **Dependency Injection / Depend on Abstraction**。

定义 `MessageSender` 接口，让 `OrderService` 通过构造函数接收依赖。

重构后 Mermaid UML



答题文字模板

该代码存在全局变量和直接依赖具体类的坏味道。`OrderService` 在内部创建 `EmailSender`，导致测试时无法替换依赖；`EmailSender` 又直接读取全局配置，形成隐藏依赖。可以引入 `MessageSender` 接口，并通过依赖注入把具体发送器传入 `OrderService`。这样业务类依赖抽象，更容易测试和扩展。

11. Comments Used to Explain Difficult Code 用注释解释复杂代码

小题背景

促销系统里有一段复杂的价格计算代码，程序员用注释解释每一步，但方法名和变量名都不清晰。

Bad Java Code

```
class PriceService {
    public double calc(Order o) {
        // calculate base price
        double x = o.getTotal();

        // apply discount if user is vip
        if (o.getCustomer().getLevel().equals("VIP")) {
            x = x * 0.8;
        }

        // add shipping fee if price is smaller than 100
        if (x < 100) {
            x = x + 10;
        }

        // return final price
        return x;
    }
}
```

指出 Bad Smell

这里存在 **Comments Used to Explain Difficult Code**，也存在 **Poor Name**。

注释不是解释业务背景，而是在解释本该由方法名和变量名表达的代码含义。

为什么不好

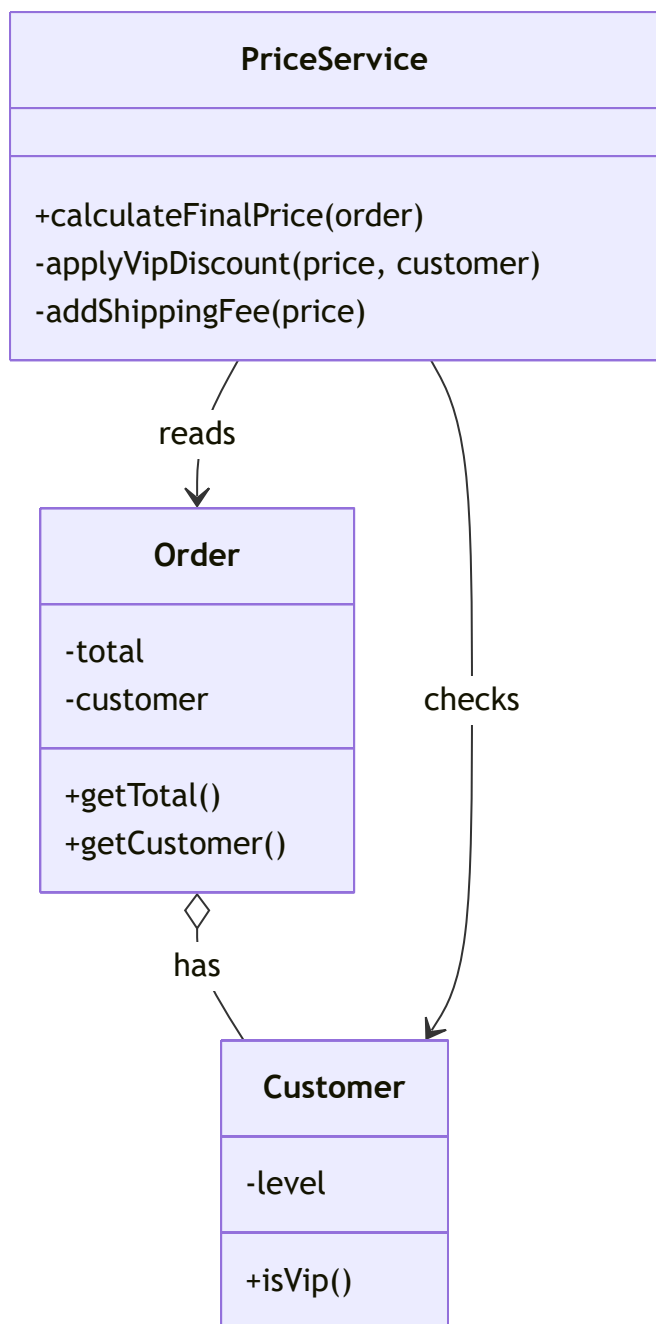
注释不能替代清晰代码。如果代码必须靠注释才能读懂，说明方法名、变量名或结构需要重构。课件中提到，不要用注释为坏代码打补丁，而应该重写代码。

重构方案

使用 **Rename Method / Rename Variable / Extract Method**。

把 `calc` 改成 `calculateFinalPrice`，把 `x` 改成 `finalPrice`，并提取折扣和运费方法。

重构后 Mermaid UML



答题文字模板

该代码存在用注释解释复杂代码的坏味道。注释说明的是代码本身应该表达的内容，说明命名和结构不清晰。可以使用 Rename Method、Rename Variable 和 Extract Method，将 `calc` 改为 `calculateFinalPrice`，并把折扣和运费逻辑提取成独立方法。重构后代码可以自解释，注释需求减少。

12. Refused Bequest 拒绝继承

小题背景

系统中 `ReadOnlyReport` 继承了 `EditableDocument`，但只需要显示功能，不支持编辑。它不得不重写 `save()`、`edit()` 并抛异常。

Bad Java Code

```

class EditableDocument {
    public void display() {
        // display document
    }

    public void edit(String text) {
        // edit document
    }

    public void save() {
        // save document
    }
}

class ReadOnlyReport extends EditableDocument {
    @Override
    public void edit(String text) {
        throw new RuntimeException("read only");
    }

    @Override
    public void save() {
        throw new RuntimeException("read only");
    }
}

```

指出 Bad Smell

这里存在 **Refused Bequest**。

子类只使用父类的一小部分功能，并拒绝继承父类的大部分行为。

为什么不好

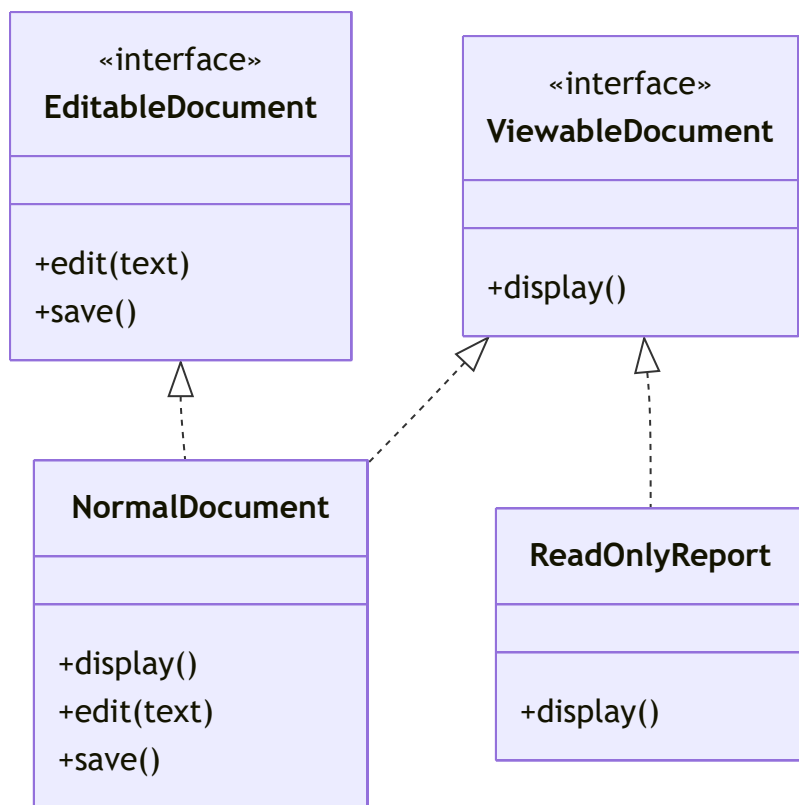
继承关系应该表示行为上的 is-a。如果 `ReadOnlyReport` 不能被当成正常的 `EditableDocument` 使用，就违反 LSP。课件中提到，如果子类只使用父类的一小部分例程，通常说明继承关系不合理。

重构方案

使用 **Extract Interface / Replace Inheritance with Delegation**。

拆分出 `ViewableDocument` 和 `EditableDocument` 两个接口。

重构后 Mermaid UML



答题文字模板

该代码存在 Refused Bequest。`ReadOnlyReport` 继承了 `EditableDocument`，但不能支持编辑和保存，只能通过抛异常拒绝父类行为，说明继承关系不合理。可以拆分接口，让只读报表只实现 `ViewableDocument`，普通文档实现可编辑接口。这样符合接口隔离原则和 LSP。

不太常见但需要了解的 Bad Smell

下面这些不一定需要单独出 UML，但考试看到时可以用文字说明。

1. Lazy Class / A class does not do very much

一个类几乎没有职责，只是简单转发或只包含很少逻辑。

重构方向：如果这个类没有存在价值，可以使用 **Inline Class**，把它合并到真正使用它的类中。

2. Speculative Generality 夸夸其谈未来性

代码中包含“以后可能会用”的接口、抽象类、配置项，但当前没有真实需求。

重构方向：删除当前不需要的抽象和死代码。不要为了猜测未来需求而增加复杂度。

3. Poor Method Name 命名不好

方法名不能说明它做什么，例如 `doIt()`、`process()`、`handle()`。

重构方向：使用 **Rename Method**，让方法名清楚表达意图。

4. Inconsistent Level of Abstraction 接口抽象层次不一致

一个类的接口里既有高层业务方法，如 `submitOrder()`，又有很低层的方法，如 `openSocket()`、`writeSql()`。

重构方向：拆分类，把高层业务接口和底层技术细节分开。

5. Parallel Inheritance Hierarchies 平行继承体系

每次给一个类增加子类，都必须给另一个类也增加对应子类。

重构方向：考虑合并层次结构，或用组合代替继承。

6. Setup / Takedown Code Around Method Call 调用前后有大量准备和收尾代码

每次调用某个方法前都要 `set` 很多字段，调用后又 `get` 很多字段。

重构方向：引入构造函数、参数对象或封装完整操作，避免反复的准备和收尾代码。

7. Shotgun Surgery 霰弹式修改

一个小需求需要修改很多类、很多文件。

重构方向：把同一变化原因相关的代码集中到一个类或模块中，减少修改扩散。

8. Data Class 数据类

类只有字段、`getter`、`setter`，没有行为。

重构方向：把和这些数据相关的行为移动到该类中，提升内聚。

常考 Bad Smell 快速识别表

代码现象	Bad Smell	常见重构
两段代码几乎一样	Duplicated Code	Extract Method / Extract Class
一个方法特别长	Long Method	Extract Method
if/for 嵌套很多层	Deeply Nested Loop	Extract Method / Guard Clause
字段 <code>public</code>	Public Data Members	Encapsulate Field
参数特别多	Long Parameter List	Introduce Parameter Object
一组参数总是一起出现	Data Clumps	Extract Class
用 <code>String/int/double</code> 表示业务概念	Primitive Obsession	Replace Primitive with Object

代码现象	Bad Smell	常见重构
一个方法总访问另一个类的数据	Feature Envy	Move Method
一个类什么都管	Large Class / Poor Cohesion	Extract Class
到处 if/switch 判断类型	Switch Statements	Replace Conditional with Polymorphism / Strategy
多个 if/switch 要一起改	Parallel Case Statements	Strategy / State / Polymorphism
一个类知道另一个类太多内部结构	Inappropriate Intimacy	Encapsulate Field / Hide Delegate
到处用全局变量	Global Variables	Encapsulate Global / DI
用注释解释复杂代码	Comments as Deodorant	Extract Method / Rename
子类拒绝父类行为	Refused Bequest	Replace Inheritance with Delegation / Extract Interface
类几乎没事做	Lazy Class	Inline Class
为未来需求写了一堆现在不用的代码	Speculative Generality	Remove Dead Code
改一个需求要改很多类	Shotgun Surgery	Move Method / Extract Class